VIJETH BALASUBRAMANIAM - 201664207

Question 1:

K-means Clustering Algorithm

A well-liked unsupervised learning approach for grouping n-dimensional points into k groups is the K-means clustering algorithm. According on how similar the data points are to one another, this technique divides the data into k groups. With K-means, the total squared distance between each data point and the cluster mean to which it is assigned is minimised. The method involves recalculating the centroids based on the newly allocated data points after iteratively assigning data points to the closest cluster centroid. The algorithm is complete after the assignments remain constant or after a predetermined number of iterations.

Algorithm for K-means Clustering:

Step 1: Determining K, the number of clusters.

Step 2: Randomly select K data points and assign each one to a cluster.

Step 3: Cluster centroids will be calculated in step three.

Step 4: Repeat the previous steps until the ideal centroid is reached, which is the centroid's data point that does not change.

- Calculated is the sum of the squared distances between the centroids and the data points.
- Each data point should be assigned to the cluster that is closest to the others. Compute the centroids for clusters by averaging all of the cluster's data points.

Pseudocode:

- Import the essential libraries, including random, pandas, matplotlib.pyplot, and numpy.
- Create a random seed for the random and Numpy libraries.
- The word embeddings file is read, and each line is saved as a list.
- In order to store the word embeddings, create a numpy array.
- Prepare the numpy array by parsing the lines.
- Create a function called "compute silhouette coefficient" that computes the silhouette coefficient using input data and labels.
- Create a function called "k means" that accepts data, k, and max iter as inputs and clusters the data using the k-means algorithm.
- In order to obtain the labels, centroid, and silhouette coefficient, initialise k and execute the "k means" function on the word embeddings data.
- To store the silhouette coefficients, make a list of k values and an empty list.
- To obtain the silhouette coefficient for each of the k values and save it in the list, loop through the k values and call the "k means" function.
- Print the final silhouette coefficient along with the silhouette coefficients for each k value.
- Use matplotlib.pyplot to plot the silhouette coefficients for each k value.

The above algorithm is implemented in python, and uploaded as .py file

Question 2:

K-Means ++ Clustering

A variant of the K-means clustering technique is known as K-means++. With the help of this technique, the initial centres will be chosen in a way that will lead to more accurate grouping. As the initial cluster centre, one random point is chosen by the method. then, for each additional centre (up to K centres in total).

Algorithm for K-means++ Clustering:

Step 1: Decide on the centroid of the data points initially at random.

Step 2: Determine the separation between each data point and the centroid that is closest to it.

Step 3: Choose the next centroid from the data points with a probability proportional to the square of the distance from the nearest centroid that has already been chosen.

Step 4: In order to select k centroids, repeat steps 2 and 3.

Once the centroids have been seeded with k-means++, the data can be clustered using the k-means method.

Pseudocode for K-means++:

- Provide a haphazard data point as the first centroid (c1).
- If I = 2 to k, then
- Determine the distance between each data point and the predetermined closest centroid.
- Using a probability proportional to the square of how far it is from the previous centroid, choose the next centroid ci from the data points.
- Allocate the closest centroid to each data point.
- Recalculate the centroids using their allocated data points as the mean.
- Until convergence, repeat steps 3 and 4 again.

At its conclusion, the algorithm provides the complete set of cluster means. This method often outperforms the traditional K-means algorithm in terms of the quality of the clustering results, especially for high-dimensional data.

Question 3:

Bisecting k-Means Clustering

By repeatedly splitting the largest cluster into two smaller clusters, bisection k-Means clustering divides a dataset into clusters until the necessary number of clusters is obtained.

The algorithm operates as follows:

1. Create a single cluster with all of the data points in it.
2. Continue until you get the necessary number of clusters:
3. Choose the biggest cluster, first.

   a. Use k-means clustering with k=2 to apply to the chosen cluster.

   b. Calculate the sum of squared errors (SSE) for every cluster that is produced.

   c. Choose the cluster with the highest SSE, then swap it out for the recently generated clusters.

4. Send back the whole list of clusters.

Pseudocode for bisecting Kmeans Algorithm:

- Create the bisecting kmeans function, which accepts a dataset and the desired number of clusters k as parameters.
- Create a list called clusters and set its initial value to be the dataset.

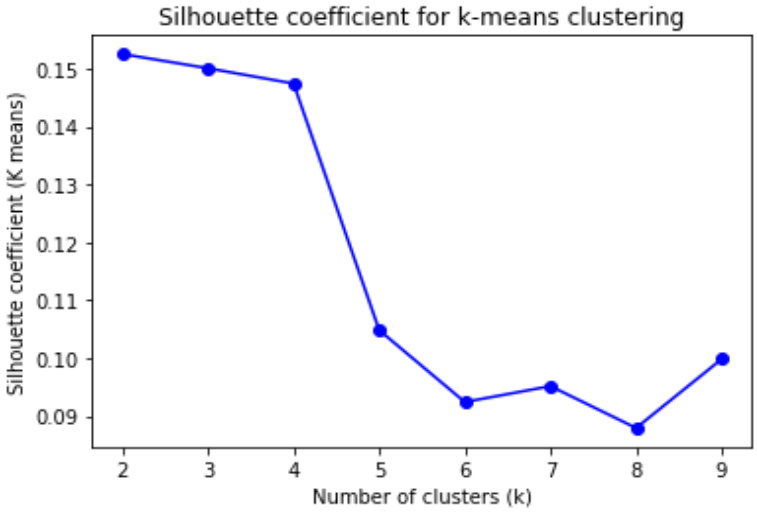
- Repeat the following steps as long as there are fewer than k clusters in each cluster:
    - Make the largest cluster the first cluster in the list of clusters.
    - Update largest cluster to reflect the cluster with the most data points after iterating through every cluster in clusters.
    - To divide largest cluster into two new clusters, smaller clusters, use the k means tool.
    - Sum of squared errors (sse) should be calculated for the two new clusters.
    - If the first new cluster's sse is higher than the second new cluster's, largest cluster must be removed from the clusters and the two new clusters must be added in that order: second new cluster first, then first new cluster.
    - If not, remove largest cluster from clusters and replace it with the first new cluster, then the second new cluster, adding the two new clusters in that order.
- Provide the final clusters list.

Question 4:

The K means clustering algorithm in the part1 is successfully executed according to the given dataset. The value of K varies from 1 to 9, 1 being the initial cluster with only 1 cluster, the code skips the value 1 and starts from 2. The silhouette coefficient for each set of clusters is plotted, as Number of clusters in X axis and Silhouette Coefficient in Y axis.

Silhouette Coefficient = (b - a) / max (a, b)

Silhouette coefficient is used to calculate the effectiveness of the clustering, with values from 1 to -1, where 1 is the best cluster clearly distinguished and -1 being the worst cluster and not clearly distinguished

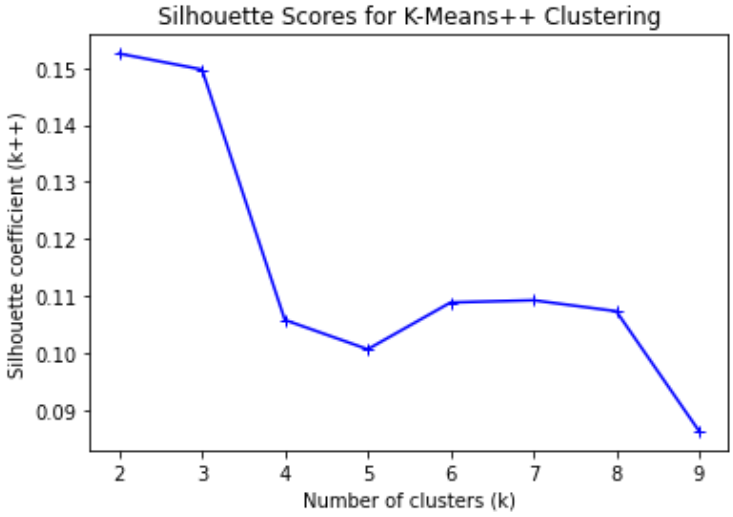


Question 5:

The K-Means++ clustering algorithm in part2 is successfully executed according to the given dataset. The value of K varies from 1 to 9, 1 being the initial cluster with only 1 cluster, the code skips the value 1 and starts from 2. The silhouette coefficient for each set of clusters is plotted, as Number of clusters in X axis and Silhouette Coefficient in Y axis.

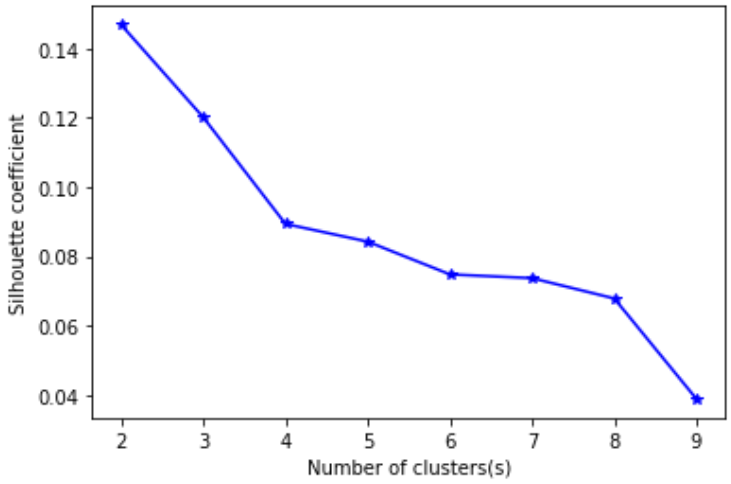
Silhouette Coefficient = (b - a) / max (a, b)

Silhouette coefficient is used to calculate the effectiveness of the clustering, with values from 1 to -1, where 1 is the best cluster clearly distinguished and -1 being the worst cluster and not clearly distinguished



Question 6:

The Bisecting k-Means algorithm implemented in part3 is successfully executed to compute a hierarchy of clustering. This is used to refine the initial single cluster to 9 clusters. We use bisecting k-means algorithm to get clusters and hierarchical structure of clusters of data points. This gives us more information than previously implemented K-Means. As mentioned in the question, the s-clusters is plotted in X axis and Silhouette coefficient in the Y axis in the same plot.



Question 7:

According to our dataset, as we know the number of clusters should be 1 to 9, the best clustering is the K-Means. From the plots we can understand that the silhouette coefficient of K-Means is **0.09937925552786803**, K-Means++ is **0.08624371476128759** and Bisecting K-Means is **0.03887456794788358.** Hence with better silhouette coefficient value K-Means is the best classifying algorithm.

K-Means:

```
k=2, Silhouette coefficient : 0.15252776165336193
k=3, Silhouette coefficient : 0.15006398638246005
k=4, Silhouette coefficient : 0.147423758718018
k=5, Silhouette coefficient : 0.10482094931688535
k=6, Silhouette coefficient : 0.0924031808506818
k=7, Silhouette coefficient : 0.09515180389059484
k=8, Silhouette coefficient : 0.08791665454977342
k=9, Silhouette coefficient : 0.09983418575630208
```

K-Means ++:

```
k++ = 2, Silhouette coefficient = 0.15252776165336193
k++ = 3, Silhouette coefficient = 0.14973462623611808
k++ = 4, Silhouette coefficient = 0.10576967867645798
k++ = 5, Silhouette coefficient = 0.10061980173449807
k++ = 6, Silhouette coefficient = 0.10883411417782618
k++ = 7, Silhouette coefficient = 0.10922433611341914
k++ = 8, Silhouette coefficient = 0.1072984074068299
k++ = 9, Silhouette coefficient = 0.08624371476128759
```

Bisecting K-Means:

```
Bisecting_k = 2, Silhouette coefficient = 0.14685763727420345
Bisecting_k = 3, Silhouette coefficient = 0.12012022882372551
Bisecting_k = 4, Silhouette coefficient = 0.08938461534644003
Bisecting_k = 5, Silhouette coefficient = 0.084278581693884
Bisecting_k = 6, Silhouette coefficient = 0.07482959233783504
Bisecting_k = 7, Silhouette coefficient = 0.07373304697060358
Bisecting_k = 8, Silhouette coefficient = 0.06787610766794135
Bisecting_k = 9, Silhouette coefficient = 0.03887456794788358
```

The above values gives detailed information of the best classifying algorithm for the dataset.